

Le Comptoir

Le client a toujours une idée de plus.

Introduction

Vous êtes développeur chez un éditeur de logiciels de caisse. Votre client est une chaîne de magasins de proximité, et il a une qualité rare : il sait exactement ce qu'il veut. Le problème, c'est qu'il le sait par petits bouts, un peu chaque jour.

Cette semaine se déroule en deux temps, et l'intention est annoncée dès maintenant, sans détour.

Temps 1, lundi et mardi. Vous développez le moteur de caisse en répondant à cinq demandes du client, dans l'ordre où elles arrivent. La consigne est explicite : ça doit marcher. La propreté, on verra après. Vous avez le droit d'aller vite, de faire des raccourcis, d'empiler des conditions. C'est le seul moment de l'année où c'est autorisé.

Temps 2, de mercredi à vendredi. Vous découvrez ce que ce code coûte. Un autre binôme l'audite, vous auditez le sien, puis chacun répare le sien. Et vendredi, une sixième demande arrive. Un code bien repris l'absorbe en vingt minutes. Un code encore emmêlé n'y arrive pas.

Personne n'est jugé sur la laideur de son code de mardi soir. Tout le monde est jugé sur ce qu'il en fait ensuite.



Cadre de travail

Composition

Binôme. L'appariement pour la phase d'audit est fixé au lancement du projet : chaque binôme audite un binôme précis et se fait auditer par un autre.

Politique IA

Interdits sur ce projet en particulier : la génération d'une classe entière, la génération d'un pattern à la demande, et surtout **la génération du rapport d'audit**. Un audit produit par une IA se repère immédiatement, il est générique et ne cite aucune ligne. Il vaut zéro.

Règle d'or de l'oral

Si vous ne savez pas expliquer une ligne de votre propre code, ou justifier un choix de pattern autrement qu'en citant un prompt, c'est éliminatoire sur la compétence concernée.

Temps 1 : le moteur de caisse

Le socle

Le logiciel gère un passage en caisse : un catalogue de produits, un panier, un total à payer. Il n'y a **ni interface graphique, ni base de données, ni framework web**. C'est de la POO pure, et c'est volontaire : rien ne doit détourner l'attention de la conception.



Le domaine minimal, à vous d'en préciser les détails :

Objet	Rôle
Product	un article du catalogue : référence, libellé, prix unitaire, catégorie
CartLine	une ligne de panier : un produit, une quantité
Cart	l'ensemble des lignes, et le calcul du total
Checkout	le point d'entrée qui produit le ticket

Rappel : **le code s'écrit en anglais**, classes, méthodes, champs et énumérations comprises. Le français est réservé à l'interface et à la documentation.

Les cinq demandes du client

Elles sont toutes listées ci-dessous. Vous les traitez **dans l'ordre**, et chacune donne lieu à un commit puis à un **tag Git imposé**, posé sur une version qui fonctionne, avant de passer à la suivante.

Demande 1, tag `v1`

Un ticket affiche la liste des lignes et le total à payer.

Demande 2, tag `v2`

Toute commande dont le total dépasse 50 euros bénéficie de 10 % de remise.

Demande 3, tag `v3`



Sur la catégorie des boissons, deux articles achetés donnent droit au troisième offert. Le moins cher des trois est celui qui est offert.

Demande 4, tag `v4`

La TVA est différenciée selon la catégorie du produit : 5,5 % sur l'alimentaire, 20 % sur le reste. Le ticket doit détailler le montant hors taxes, le montant de TVA par taux, et le total toutes taxes comprises.

Demande 5, tag `v5`

Le client introduit une carte de fidélité. Un euro dépensé rapporte un point. Cent points donnent droit à 5 euros de remise, utilisables sur une commande suivante. Et la précision qui change tout : **les remises ne se cumulent pas**. Quand plusieurs sont applicables, on retient **celle qui est la plus avantageuse pour le client**.

La règle des tags

Cette règle remplace toute intervention extérieure pendant la semaine. Elle est vérifiée à l'évaluation dans l'historique Git.

- Un tag par demande, **v1** à **v5**, posé sur un état qui compile et qui fonctionne.
- Un dépôt qui arrive vendredi avec un seul commit et cinq tags posés à la même seconde ne trompe personne, et perd les points correspondants.
- **Rien ne vous interdit de lire les cinq demandes d'avance** et de concevoir proprement dès **v1**. Ce n'est pas de la triche, c'est exactement la compétence visée. Mais le livrable reste la **suite des cinq versions fonctionnelles**, pas l'état final tout seul.

Le client de démonstration



En parallèle du moteur Java, vous écrivez un **client de démonstration en TypeScript strict** : une exécution en console ou une page minimale qui construit deux ou trois paniers et affiche les tickets obtenus.

Ce client n'est pas évalué sur son apparence. Il est évalué sur son **typage** : strict: true, aucun any, des types qui décrivent réellement le ticket et ses lignes. Il sert à prouver que vous savez modéliser le même domaine dans deux langages.

Temps 2 : l'audit croisé et la reprise

L'audit, mercredi

Vous recevez le dépôt du binôme qui vous est attribué, **en lecture seule**. Aucun push, aucune issue, aucune correction chez eux : vous écrivez un rapport, c'est tout.

Le rapport contient :

- **Les constats.** Chaque point est localisé au format fichier:ligne, classé **bloquant, majeur ou mineur**, et justifié par le principe violé. « Cette classe change pour deux raisons différentes » est un constat. « Je trouve ça pas très propre » n'en est pas un.
- **Les points de rupture.** Où le code cassera-t-il à la prochaine évolution, et pourquoi.
- **La sixième demande client.** Voir ci-dessous.

La sixième demande, écrite par vous

C'est la dernière section obligatoire de votre rapport d'audit. Vous rédigez **une nouvelle demande client que le code audité, selon vous, encaissera mal**.

Le cadre, à respecter sous peine de voir la demande refusée :



- elle porte sur les règles de prix, de remise ou de fidélité, **pas sur une nouvelle technologie** ;
- elle tient en cinq lignes maximum ;
- elle doit être implémentable en **deux heures dans un code correctement conçu** ;
- elle ne remet en cause aucune des cinq demandes précédentes.

Vous êtes évalués sur la **pertinence** de cette demande. Pour écrire une bonne demande piège, il faut avoir compris où le code est rigide : c'est la vraie preuve que votre audit était sérieux.

La réponse à l'audit reçu, jeudi matin

Vous recevez l'audit qui vous concerne. Pour chaque point, vous répondez par écrit : **corrigé**, ou **contesté** avec un argument technique.

Contester avec raison vaut autant que corriger. Un auditeur peut se tromper, et savoir défendre un choix de conception fait partie du métier. En revanche, contester sans argument est compté comme un point non traité.

Le refactoring, jeudi et vendredi

Vous reprenez votre propre code sur une branche `refactor`, une pull request par thème, avec une description qui explique l'avant et l'après.

Exigences :

- **Au moins deux design patterns effectivement implémentés**, pas décrits dans un document. Le domaine les appelle naturellement : le calcul des remises est un cas d'école de **Strategy**, l'empilement des règles se prête à une composition ou à une **Chain of Responsibility**, la construction des lignes de ticket à une **Factory**, le cumul des points de fidélité à un **Observer**.
- **Zéro régression**. Les cinq demandes fonctionnent toujours à l'identique. Vous devez pouvoir le prouver en rejouant vos scénarios de démonstration.



- Un **diagramme UML avant et après** pour au moins un des deux patterns.

L'épreuve du vendredi après-midi

Vous implémentez la sixième demande reçue, sur une branche dédiée, taguée v6.

C'est le moment de vérité de la semaine, et il ne se discute pas : soit votre conception encaisse le changement, soit elle ne l'encaisse pas.

Livrables

1. **Dépôt Git dont l'historique porte les tags v1 à v5, chacun sur une version fonctionnelle.**
2. **Moteur de caisse en Java 21 et client de démonstration en TypeScript strict.**
3. **Rapport d'audit du dépôt partenaire, 3 pages maximum, constats localisés et priorisés, terminé par la sixième demande client.**
4. **Réponse écrite à l'audit reçu, point par point, corrigé ou contesté.**
5. **Code refactorisé, avec au moins deux patterns implémentés et un diagramme UML avant et après.**
6. **Sixième demande reçue implémentée et taguée v6.**
7. **README avec instructions de build et d'exécution**



Compétences visées

- **BC01-C3** : développer des composants métier, niveau Analyser
- **BC01-C4** : contribuer à la gestion d'un projet informatique, par la revue de code
- **BC02-C2** : définir l'architecture logicielle d'une application

Rendu

Le projet est à rendre sur un dépôt GitHub par binôme :

<https://github.com/prenom-nom/LeComptoir>

Structure attendue :

LeComptoir/

engine/ (moteur Java 21)

demo/ (client de démonstration TypeScript)

docs/ (audit produit, réponse à l'audit reçu, UML)

README.md



Base de connaissances

Conception

- [Principes SOLID, présentation d'origine par Robert C. Martin](#)
- [Refactoring Guru, catalogue des design patterns](#)
- [Refactoring Guru, catalogue des code smells](#)

Java

- [Java 21, documentation officielle](#)
- [Records Java](#)
- [Sealed classes](#)

TypeScript

- [TypeScript Handbook](#)
- [Options de compilation strictes](#)

Git

- [Tags Git](#)
- [Conventional Commits](#)
- [Pull requests GitHub](#)